



Maven Setup

Build | Manage | Automate | Simplify

Set Up Maven | Build Projects | Manage Dependencies | Run with Ease

Simple
Builds
Powerful
Applications



What is Maven?

- Maven is a build automation tool used primarily for Java projects.
- It helps in building, managing dependencies, compiling code, testing, and packaging applications.
- It uses a project object model (POM) file (pom.xml) to manage the project configuration.
- Maven follows a standard project structure and build lifecycle.



Key Features of Maven

- 1 Simplifies project build and management.
- 2 Manages dependencies automatically (from central repository).
- 3 Uses a standard project structure.
- 4 Provides a consistent build lifecycle.
- 5 Easy integration with IDEs (Eclipse, IntelliJ, VS Code).
- 6 Supports plugins for various tasks.
- 7 Widely used in enterprise and open source projects.



Steps to Setup Maven

- 1 **Download Maven**
Download the latest version from the official Apache Maven website.
- 2 **Extract the Files**
Extract the downloaded zip file to a folder (e.g., C:\Program Files\Apache\maven).
- 3 **Set Environment Variables**
Set MAVEN_HOME and update the PATH variable.
- 4 **Verify Installation**
Open command prompt and run:
`mvn -version`



Environment Variables

Variable	Value (Example)
MAVEN_HOME	C:\Program Files\Apache\maven-3.9.6
Path	%MAVEN_HOME%\bin



Verify Maven Installation

```
C:\> mvn -version
Apache Maven 3.9.6
Maven home: C:\Program Files\Apache\maven-3.9.6
Java version: 17.0.8, vendor: Oracle Corporation
Default locale: en_US, platform encoding: UTF-8
OS name: "Windows 10", version: "10.0", arch: "amd64"
```



Maven Installed
Successfully!



Project Structure

```
my-maven-project/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   └── resources/
│   └── test/
│       ├── java/
│       └── resources/
```



Standard
Maven Project
Structure



Sample pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-maven-project</artifactId>
  <version>1.0-SNAPSHOT</version>
</project>
```



Useful Maven Commands

Command	Description
<code>mvn clean</code>	Cleans the target directory
<code>mvn compile</code>	Compiles the source code
<code>mvn test</code>	Runs the tests
<code>mvn package</code>	Packages the application (creates .jar/.war)
<code>mvn install</code>	Installs the package to local repository
<code>mvn spring-boot:run</code>	Runs a Spring Boot application



Why Use Maven?

- ✓ Automates the build process
- ✓ Manages dependencies easily
- ✓ Saves time and reduces manual effort
- ✓ Standard and widely adopted tool
- ✓ Works well with modern Java frameworks (Spring Boot, etc.)

“Write Less
Build More”



BUILD TODAY | DEPLOY TOMORROW | WITH MAVEN



Better Builds
Brighter Future



Introduction to Maven

Apache Maven is a build automation and project management tool mainly used for Java applications.

Maven helps us automate common tasks such as:

- Creating a standard project structure
- Compiling Java source code
- Managing external libraries/dependencies
- Running tests
- Packaging applications into **JAR/WAR** files
- Running applications
- Managing project information
- Supporting different build environments

In simple words:

Maven manages your Java project's dependencies and automates the build process.

Why Do We Need Maven?

Suppose you create a Java application that uses:

- ✓ MySQL
- ✓ JUnit
- ✓ Spring
- ✓ Hibernate



Without Maven, you may need to:

- ✓ Download each .jar file manually.
- ✓ Add the JARs to the project.
- ✓ Find compatible versions.
- ✓ Update JARs manually.
- ✓ Configure the compiler.
- ✓ Compile the project manually.
- ✓ Run tests manually.
- ✓ Create the final JAR manually.

Maven automates most of this.

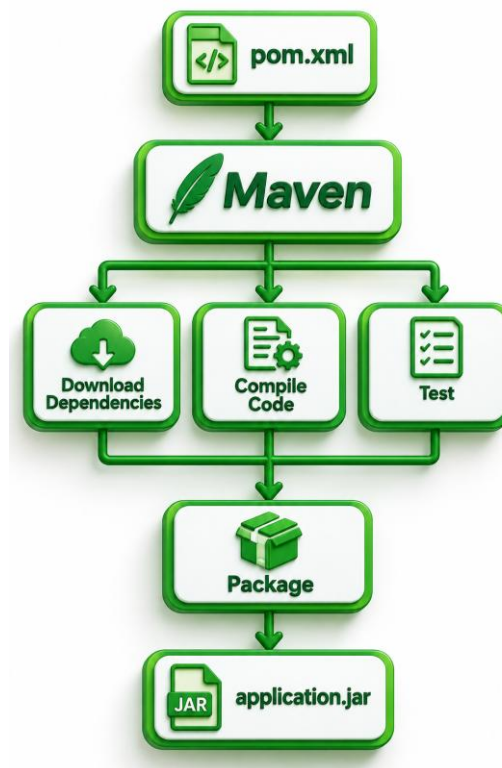
For example, instead of manually downloading **MySQL Connector/J**, we can write:

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>9.4.0</version>
</dependency>
```

Maven downloads the required library automatically.

Maven Architecture

A simplified Maven workflow is:



What is pom.xml?

The most important file in a Maven project is:

pom.xml

POM stands for: Project Object Model

It contains information about the project, such as:

- ✓ Project name
- ✓ Group ID
- ✓ Artifact ID
- ✓ Version

- ✓ Java version
- ✓ Dependencies
- ✓ Plugins
- ✓ Build configuration

Example:

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>

</project>
```

Understanding Maven Coordinates

Three important elements are:

XML

```
<groupId>com.example</groupId>
<artifactId>my-app</artifactId>
<version>1.0-SNAPSHOT</version>
```

- ✓ **groupId** – Identifies the organization or package/group.

`<groupId>com.example</groupId>`

For a company:

`<groupId>com.futuretech</groupId>`

- ✓ **artifactId** – Identifies the project/application.

`<artifactId>student-management</artifactId>`

- ✓ **version** – Specifies the project version.

`<version>1.0.0</version>`

Therefore:

Group ID = com.example

Artifact ID = my-app

Version = 1.0-SNAPSHOT (You can change it like 1.0.0)

Together they uniquely identify a Maven artifact.

Maven Features

The Maven Build Automation tool provides a lot of features to make the development easy. Below we listed them

Dependency Management:

- ✓ Maven handles external libraries (dependencies) automatically by downloading them from a central repository (Maven Central).
- ✓ Example: Instead of manually downloading a JAR, you just declare it in `pom.xml`.

Build Automation:

Compile code, run tests, package JAR/WAR files, and deploy them—all using simple commands.

- ✓ mvn compile
- ✓ mvn test
- ✓ mvn package
- ✓ mvn install

Plugin and Goals:

Maven uses plugins to perform tasks. Each plugin has “goals” that can be executed.

Example: maven-compiler-plugin compiles Java source code.

Transitive Dependencies:

- ✓ Maven automatically includes dependencies of your dependencies

Lifecycle Phases:

In Maven Build life cycle there are different phases these phases are used for different purposes like test phase for compiles tests, verify phases is used for tests and main source code and other.

Maven defines a default build lifecycle, with phases such as:

- ✓ **Validate** – This phase is responsible for validates if the project structure is correct or not.
- ✓ **Compile** – It compiles the source code, converts the `.java` files to `.class`, and stores these classes in the target/classes folder.
- ✓ **Test** – It runs unit tests for the maven project.

- ✓ **Package** – This phase is responsible for distribute complied code in the format of WAR, JAR and others.
- ✓ **Verify** – verify that the project is valid and meets the quality standards.
- ✓ **Install** – This phase is responsible for install packaged code on the system.
- ✓ **Deploy** – Copies the packaged code to the remote repository for deployment then other developers can easily access this one.

Maven Configuration (Setup Maven)

Download Apache Maven:

<https://maven.apache.org/download.cgi>

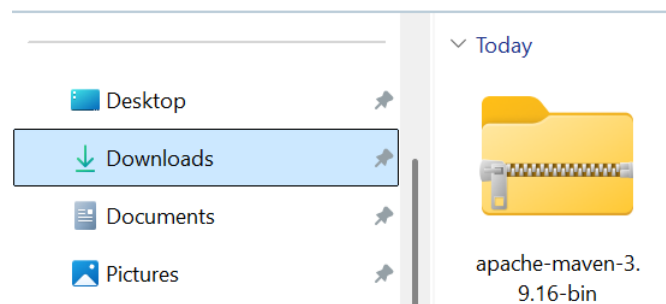
Download Maven Binary zip archive in the Files section

	Link
Binary tar.gz archive	apache-maven-3.9.16-bin.tar.gz
Binary zip archive	apache-maven-3.9.16-bin.zip
Source tar.gz archive	apache-maven-3.9.16-src.tar.gz
Source zip archive	apache-maven-3.9.16-src.zip

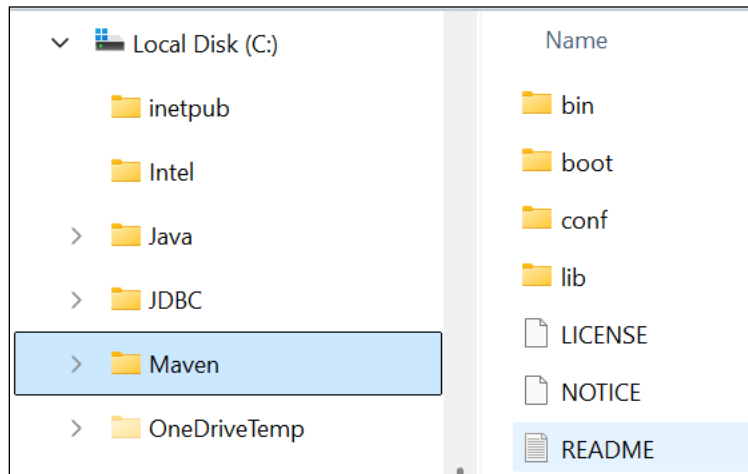
After Downloading

Location of downloaded **zip** is **Downloads** folder

- ✓ Open the Downloads folder
- ✓ Downloaded zip looks like
- ✓ Extract the folder
- ✓ Paste the folder at specific location



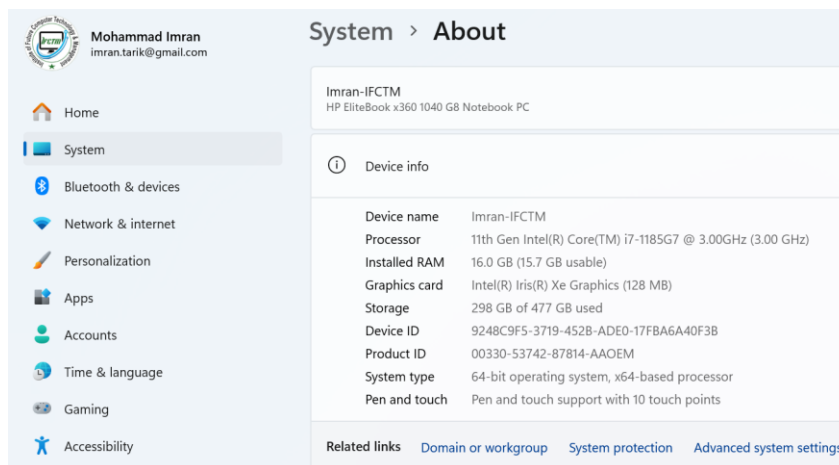
- ✓ Like **C:\Maven\apache-maven-3.9.16-bin**
- ✓ Content of the **C:\Maven** folder will look like



Now, set the path for maven using environment variable

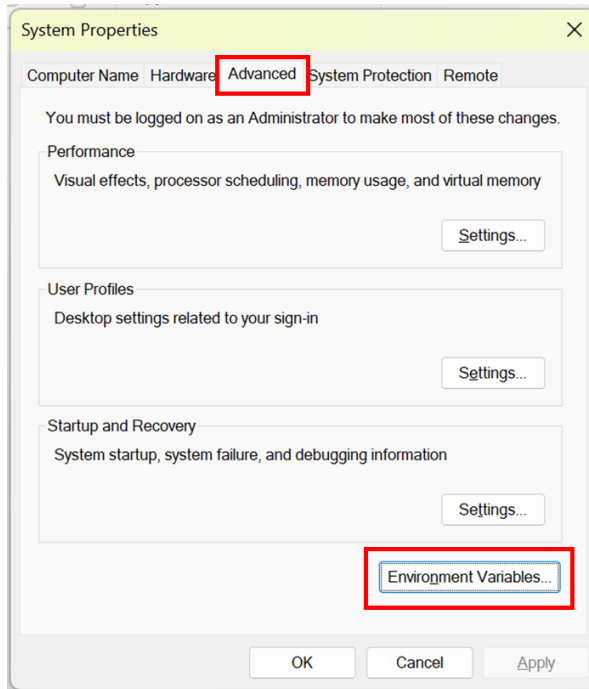
To Open **System Properties** dialog box

- ✓ Open Setting

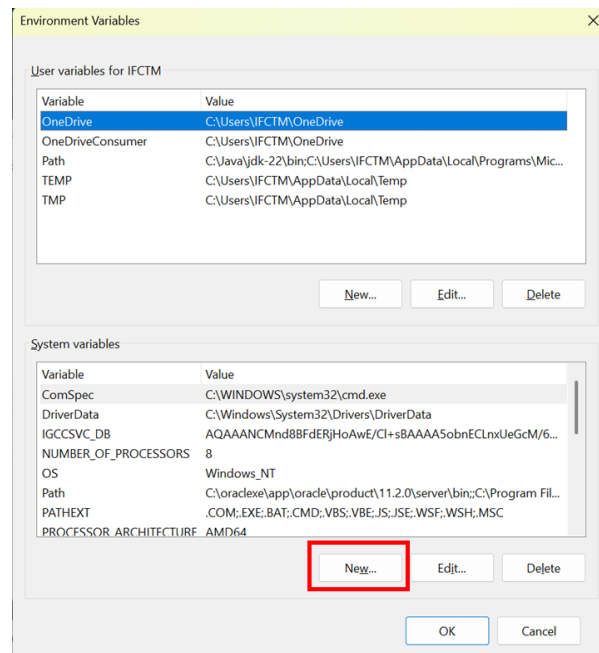


- ✓ Click on System
- ✓ Click on About
- ✓ Click on Advance System Setting

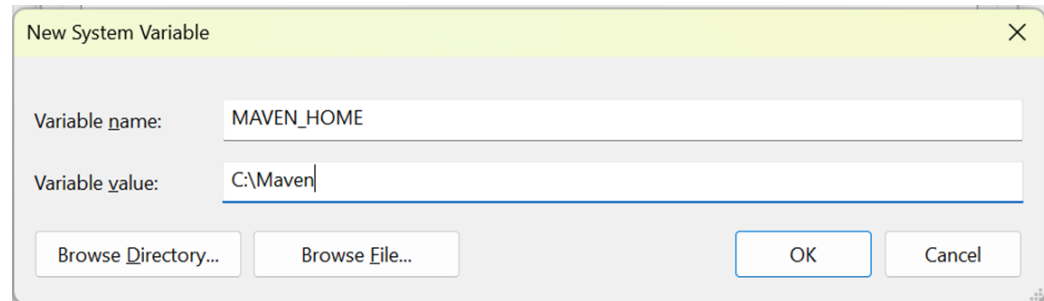
System Properties dialog box is displayed



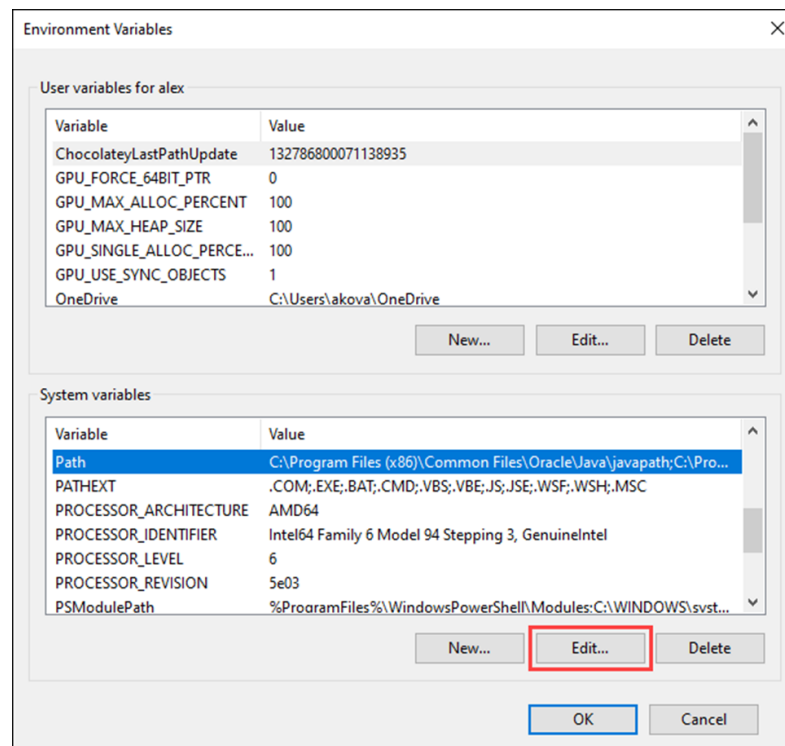
- ✓ In System Properties dialog box
- ✓ Click on Advance Tab
- ✓ Now, Click on Environment Variables button
- ✓ **Environment Variables** window displayed



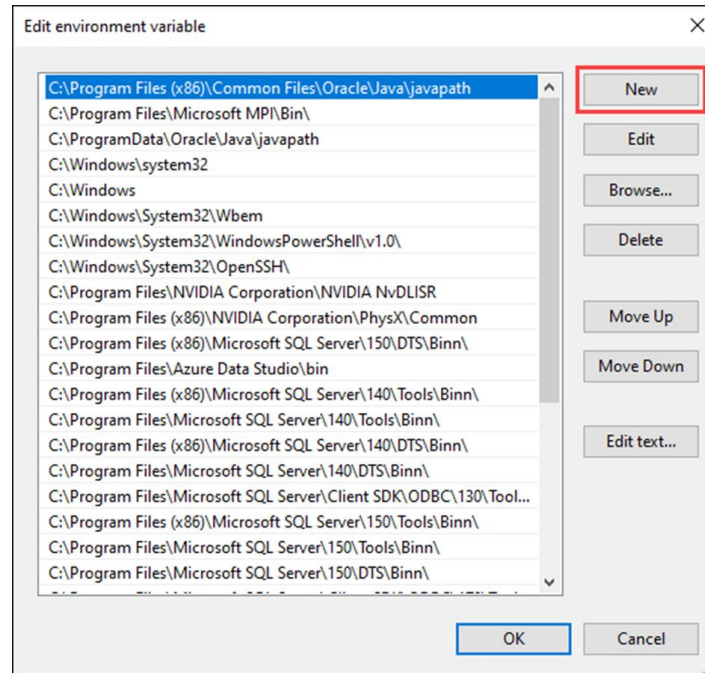
- ✓ In **System variables** section, Click on **New**
- ✓ In the **New System Variable** window displayed



- ✓ In this window
- ✓ Set **Maven Home** path
 - Variable name - MAVEN HOME
- ✓ Set Maven Home location
 - Variable value - C:\Maven (It will be according to your location)
- ✓ Click **Ok**
- ✓ Select the **Path** variable under the **System variables** section in the **Environment Variables** window.



- ✓ Click on **Edit**
- ✓ Now, **Edit environment variable** dialog box is displayed



- ✓ Click on **New** button
- ✓ Type **C:\Maven\bin**
- ✓ Click **Ok**

After completing configuration

- ✓ Check **Java** and **Maven** is working fine
- ✓ Open **Command Prompt**
- ✓ Type **java -version** and **mvn -version**

```

Microsoft Windows [Version 10.0.26100.4770]
(c) Microsoft Corporation. All rights reserved.

C:\Users\IFCTM>java -version
java version "22.0.2" 2024-07-16
Java(TM) SE Runtime Environment (build 22.0.2+9-70)
Java HotSpot(TM) 64-Bit Server VM (build 22.0.2+9-70, mixed mode, sharing)

C:\Users\IFCTM>mvn -version
Apache Maven 3.2.11 (2013-09-11T13:42:22Z; 4957b63ee3494413a2b544fd3d825b)
Maven home: C:\Maven
Java version: 22.0.2, vendor: Oracle Corporation, runtime: C:\Java\jdk-22
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

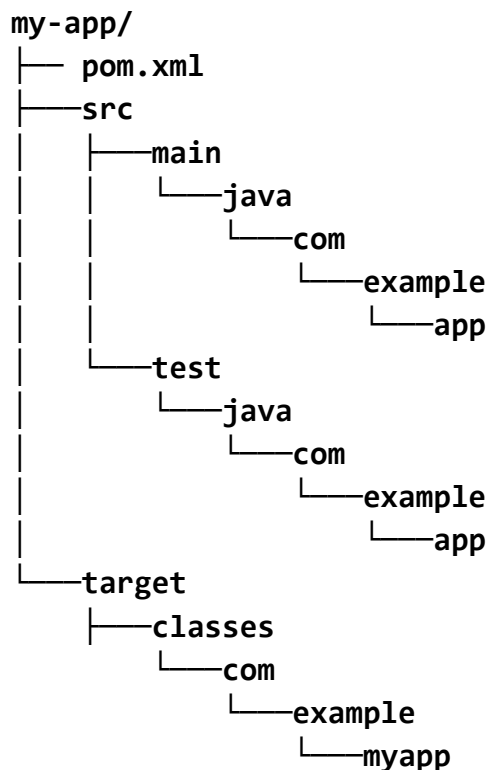
C:\Users\IFCTM>
  
```

Task Performed by Build Tool

Task	What it Does
Compile	Translates .java to .class files
Package	Bundles .class files into .jar, .war, etc.
Dependency Management	Automatically downloads and links libraries
Testing	Runs unit or integration tests
Clean	Deletes old builds
Deploy	Uploads the final artifact to a server or repo

Maven Standard Project Structure

Maven directory structure – Without .jar



```

✓ first-app
  > .github
  ✓ src
    ✓ main\java\com\example\app
      J App.java
    ✓ test\java\com\example\app
      J AppTest.java
  ✓ target
    > classes
    > generated-sources
    > maven-status
    > test-classes
  pom.xml
  
```

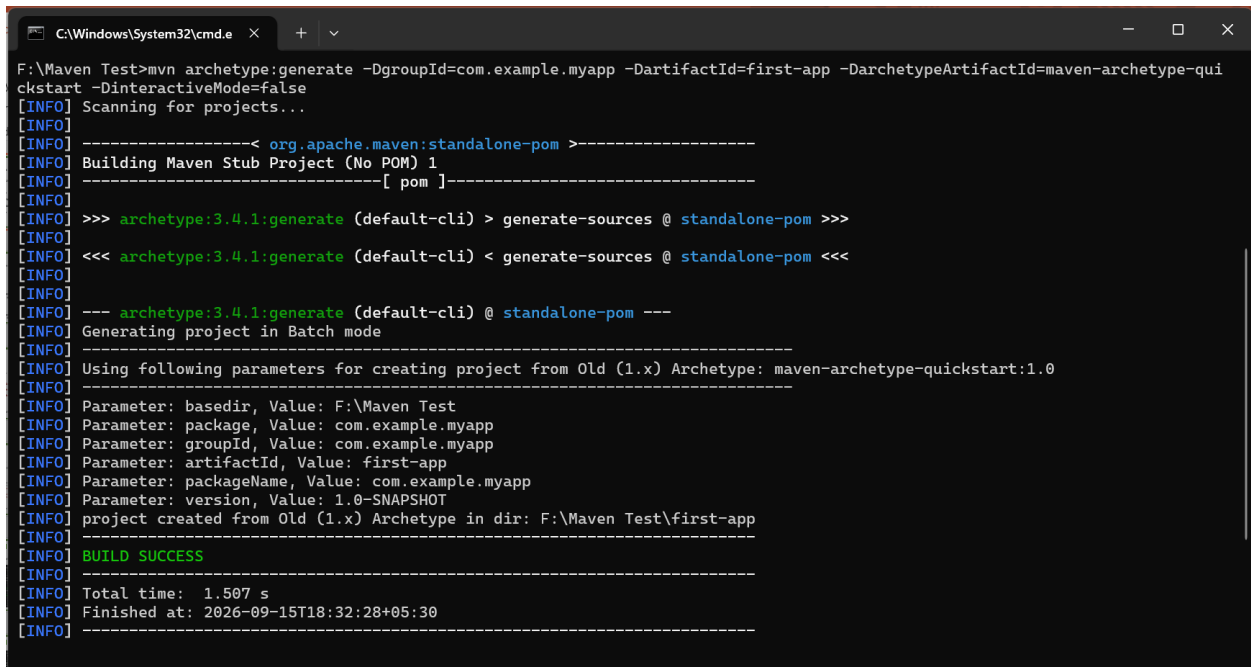
NOTE: `mvn` command will create this structure automatically.

Example – 1

Step – 1

Write mvn command

```
mvn archetype:generate -DgroupId=com.example.myapp -
-DartifactId=first-app -DarchetypeArtifactId=maven-archetype-
quickstart -DinteractiveMode=false
```



```
C:\Windows\System32\cmd.exe
F:\Maven Test>mvn archetype:generate -DgroupId=com.example.myapp -DartifactId=first-app -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> archetype:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< archetype:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- archetype:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] -----
[INFO] Parameter: basedir, Value: F:\Maven Test
[INFO] Parameter: package, Value: com.example.myapp
[INFO] Parameter: groupId, Value: com.example.myapp
[INFO] Parameter: artifactId, Value: first-app
[INFO] Parameter: packageName, Value: com.example.myapp
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: F:\Maven Test\first-app
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.507 s
[INFO] Finished at: 2026-09-15T18:32:28+05:30
[INFO]
[INFO] -----
```

Explanation of command:

Parameter	Description
-DgroupId	Java package-style ID for the project group (e.g., com.example)
-DartifactId	Name of the project directory and artifact (e.g., my-app)
-DarchetypeArtifactId	Archetype template to use (e.g., maven-archetype-quickstart)
-DinteractiveMode=false	Disable prompts so the command runs automatically

Step – 2

Pom.xml looks like

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example.myapp</groupId>
  <artifactId>first-app</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>first-app</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

App.java File

```
package com.example.myapp;

/**
 * Hello world!
 *
 */
public class App
```

```

{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );
    }
}

```

Step – 3

To Compile – mvn compile

```

C:\Windows\System32\cmd.e
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example.myapp:first-app >-----
[INFO] Building first-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ first-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory F:\Maven Test\first-app\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ first-app ---
[INFO] Recompiling the module because of changed source code.
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 1.8] to target\classes
[WARNING] bootstrap class path is not set in conjunction with -source 8
[WARNING] not setting the bootstrap class path may lead to class files that cannot run on JDK 8
--release 8 is recommended instead of -source 8 -target 1.8 because it sets the bootstrap class path automatically
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.532 s
[INFO] Finished at: 2026-09-15T18:43:04+05:30
[INFO]
F:\Maven Test\first-app>

```

Step – 4

To Run Code – java -cp target\classes com.example.myapp.App

```

C:\Windows\System32\cmd.e
F:\Maven Test\first-app>java -cp target\classes com.example.myapp.App
Hello World!
F:\Maven Test\first-app>

```

Output: Hello World!

Example – 2

To make your **Maven project produce an executable .jar** file that runs on double-click, you need to configure the Maven JAR plugin (or the Shade plugin) to include a Main-Class entry in the manifest. That way, Java knows which class to run when the JAR is launched.

Step – 1

Write mvn command

```
mvn archetype:generate -DgroupId=com.example.myapp -  
DartifactId=calculator-executable-jar -  
DarchetypeArtifactId=maven-archetype-quickstart -  
DinteractiveMode=false
```

Step – 2

Update the **pom.xml** and add **maven-jar-plugin** configuration under <build>:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
http://maven.apache.org/maven-v4_0_0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.example.myapp</groupId>  
  <artifactId>calculator-executable-jar</artifactId>  
  <packaging>jar</packaging>  
  <version>1.0-SNAPSHOT</version>  
  <name>calculator-executable-jar</name>  
  <url>http://maven.apache.org</url>  
  <dependencies>  
    <dependency>
```

```
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
<build>
  <plugins>
    <!-- Add this plugin -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.3.0</version>
      <configuration>
        <archive>
          <manifest>
            <!-- Point to your main class -->
            <mainClass>com.example.myapp.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

Step – 2

```
package com.example.myapp;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class App implements ActionListener
{
    JFrame frame;
    JPanel panel;
    JTextField t1, t2, tresult;
    JButton addBtn, subBtn, mulBtn, divBtn, exitBtn;

    public App()
    {
        frame = new JFrame("Simple Calculator");
        frame.setSize(500, 350);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        panel = new JPanel();

        // Input fields
        JLabel l1 = new JLabel("First Number:");
        t1 = new JTextField(10);

        JLabel l2 = new JLabel("Second Number:");
        t2 = new JTextField(10);

        JLabel l3 = new JLabel("Result:");
        tresult = new JTextField(10);
        tresult.setEditable(false);
    }
}
```

```
// Buttons
addBtn = new JButton("ADD");
subBtn = new JButton("SUBTRACT");
mulBtn = new JButton("MULTIPLY");
divBtn = new JButton("DIV");
exitBtn = new JButton("EXIT");

panel.setLayout(null); // <--- IMPORTANT

//setBounds for Layouts
l1.setBounds(50, 50, 100, 25); // <--- positioning
t1.setBounds(160, 50, 150, 25);
l2.setBounds(50, 100, 120, 25);
t2.setBounds(160, 100, 150, 25);
l3.setBounds(50, 150, 100, 25);
tresult.setBounds(160, 150, 150, 25);
addBtn.setBounds(50, 200, 100, 25);
subBtn.setBounds(160, 200, 120, 25);
mulBtn.setBounds(290, 200, 120, 25);
divBtn.setBounds(50, 250, 100, 25);
exitBtn.setBounds(160, 250, 100, 25);

// Add listeners
addBtn.addActionListener(this);
subBtn.addActionListener(this);
mulBtn.addActionListener(this);
divBtn.addActionListener(this);
exitBtn.addActionListener(this);

// Add components to panel
panel.add(l1); panel.add(t1);
panel.add(l2); panel.add(t2);
```

```
panel.add(l3); panel.add(tresult);
panel.add(addBtn); panel.add(subBtn);
panel.add(mulBtn); panel.add(divBtn);
panel.add(exitBtn);

frame.add(panel);
frame.setVisible(true);
}

@Override
public void actionPerformed(ActionEvent e)
{
    if (e.getSource() == exitBtn)
    {
        System.exit(0);
    }
    try
    {
        double num1 = Double.parseDouble(t1.getText());
        double num2 = Double.parseDouble(t2.getText());
        double result;

        if (e.getSource() == addBtn)
        {
            result = num1 + num2;
            tresult.setText(String.valueOf(result));
        }
        if (e.getSource() == subBtn)
        {
            result = num1 - num2;
            tresult.setText(String.valueOf(result));
        }
    }
}
```

```

        if (e.getSource() == mulBtn)
        {
            result = num1 * num2;
            tresult.setText(String.valueOf(result));
        }
        if (e.getSource() == divBtn)
        {
            if (num2 != 0)
            {
                result = num1 / num2;
                tresult.setText(String.valueOf(result));
            }
            else
            {
                tresult.setText("Error: Divide by 0");
                //return;
            }
        }
    }
    catch (NumberFormatException ex)
    {
        tresult.setText("Invalid Input");
    }
}

public static void main(String[] args)
{
    new App();
}
}

```

Step – 4

Build the JAR – Run in terminal:

```
mvn clean
```

```
mvn compile
```

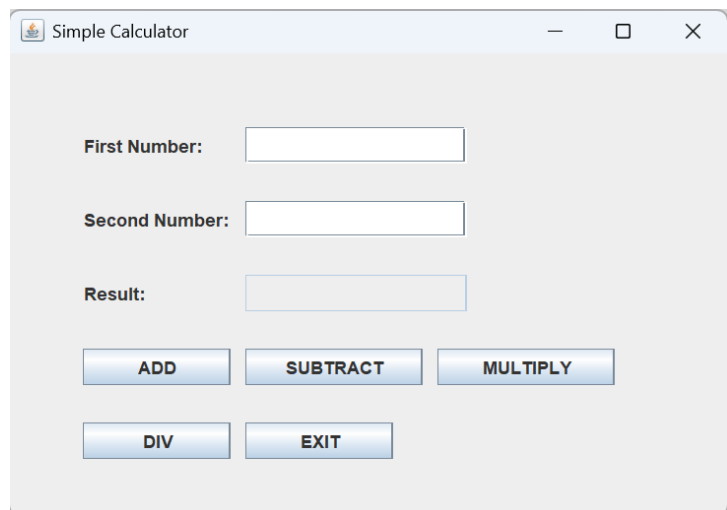
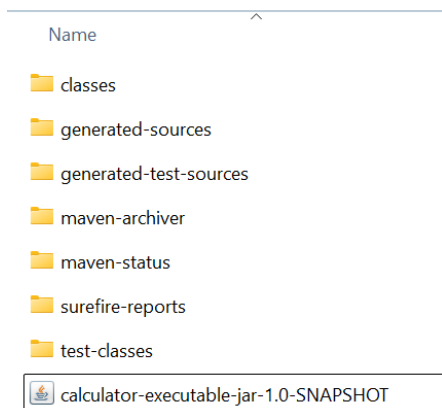
```
mvn package //To build a .jar
```

Step – 5

Run the JAR – You can now run it directly:

```
java -jar target/calculator-executable-jar-1.0-SNAPSHOT.jar
```

Or simply double-click the **JAR** file in your file explorer — because the manifest specifies **com.example.myapp.App** as the entry point, it will launch your Swing calculator. (target folder in project folder).



After double click on .jar

file app is displayed.



What is Maven Repository?

Maven uses repositories to store and retrieve dependencies.

There are three important types.

✓ **Local Repository**

Located on your computer.

Typically:

`C:\Users\<username>\.m2\repository`

Once Maven downloads a dependency, it can reuse it from the local repository.

✓ **Central Repository**

The **Maven Central Repository** is the primary public repository containing Java libraries.

For example:

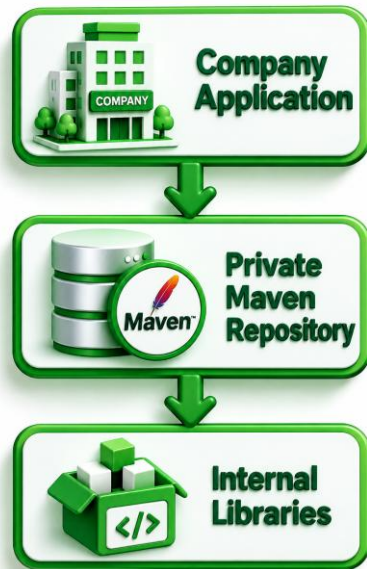
JUnit
Spring
Hibernate
Jackson
MySQL Connector

Maven can download dependencies from Central automatically.

✓ **Remote/Private Repository**

Organizations can maintain their own repositories for internal libraries.

For example:



Maven Life Cycle

Maven has predefined build lifecycles.

The most commonly used lifecycle is:



- ✓ **Validate** – Checks whether the project configuration is correct.

```
mvn validate
```

- ✓ **Compile** – Compiles Java source code.

```
mvn compile
```

- ✓ **Test** – Runs unit tests.

```
mvn test
```

- ✓ **Package** – Creates JAR/WAR.

```
mvn package
```

- ✓ **Install** – Installs the project into your local Maven repository.

```
mvn install
```

For Example:

```
my-app-1.0-SNAPSHOT.jar
```

↓

```
.m2/repository
```

- ✓ **Deploy** – Publishes the artifact to a remote repository.

```
mvn deploy
```

Maven Plugin

Maven uses plugins to perform tasks.

For example:



- ✓ The Maven Compiler Plugin compiles Java code.
- ✓ The Maven Surefire Plugin runs tests.
- ✓ The Maven JAR Plugin creates JAR files.

Maven with Java Version

You can specify the Java version in pom.xml.

For example, Java 22:

```
<properties>
  <maven.compiler.source>22</maven.compiler.source>
  <maven.compiler.target>22</maven.compiler.target>
</properties>
```

A modern project might instead use:

```
<properties>
  <maven.compiler.release>22</maven.compiler.release>
</properties>
```

This tells Maven to compile the project for Java 22.

Platform Encoding Warning

Maven is saying that it is using the system's encoding when processing resources.

To avoid the warning and make the build consistent across different computers, you can specify UTF-8 explicitly.

Add this to your **<properties>**:

```
<properties>
  <maven.compiler.release>22</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8
</project.reporting.outputEncoding>
</properties>
```

```
C:\Windows\System32\cmd.exe X + -
D:\SRM Maven Practical\CORE-B\dependency-injection-ioc>mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:dependency-injection-ioc >-----
[INFO] Building dependency-injection-ioc 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ dependency-injection-ioc ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ dependency-injection-ioc ---
[INFO] Recompiling the module because of changed source code.
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 5 source files with javac [debug target 1.8] to target\classes
[WARNING] bootstrap class path is not set in conjunction with -source 8
[WARNING] not setting the bootstrap class path may lead to class files that cannot run on JDK 8
--release 8 is recommended instead of -source 8 -target 1.8 because it sets the bootstrap class path automatically
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.849 s
[INFO] Finished at: 2026-09-16T12:49:27+05:30
[INFO] -----
```

Maven in Spring Boot

Maven is especially common with Spring Boot.

A Spring Boot project typically contains:

```
spring-app
|
├─ pom.xml
|
└─ src
    ├─ main
    │   ├─ java
    │   └─ resources
    └─ test
        └─ java
```

The **pom.xml** contains Spring Boot dependencies such as:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Then we can run:

```
mvn spring-boot:run
```

Conclusion

Maven is an important tool for Java development that simplifies project building, dependency management, testing, and packaging. It uses the pom.xml file to manage project configuration, libraries, plugins, and build settings.



By using Maven commands such as `mvn compile`, `mvn test`, `mvn package`, and `mvn clean`, developers can automate the complete build process instead of performing these tasks manually.

Maven's standard project structure also makes Java projects easier to develop, maintain, build, and share. It is widely used in modern Java applications, especially Spring Boot and enterprise applications.

Therefore, understanding POM, dependencies, repositories, lifecycle phases, and plugins provides a strong foundation for working with Java and Spring Boot projects.